Multi Object Editor - 複数オブジェクト編集コンポーネント

概要

目的

Creator、Vendor、Owner、Instrumentなどの複数オブジェクトを登録・編集するための汎用コンポーネントです。追加/削除/並び替え機能、UUID自動生成、参照プロパティ対応、マスタデータからの取り込み機能を提供します。

使用場面

- DocumentEditDialog内でCreators/Vendors/Owners/Instrumentsを編集する時
- MAIML Documentセクションで複数の関連オブジェクトを管理する時
- マスタデータから既存定義を取り込む時

アクセス権限

- **実験管理者（Experiment Admin）**: 利用可能
- MAIML編集権限を持つユーザーのみアクセス可能

コンポーネント構成

このコンポーネントは2つのサブコンポーネントで構成されています：

1. MultiObjectEditor（メインコンポーネント）

複数オブジェクトの一覧管理と追加機能を提供

2. SingleObjectEditor（個別編集コンポーネント）

個別のオブジェクト詳細編集と並び替え機能を提供

画面構成

レイアウト構造（MultiObjectEditor）



レイアウト構造（SingleObjectEditor - マスタから追加時）

```
┌──────────────────────────────────────────────────────┐
│ [Collapse] #N [Name] [ID: xxx]         [↑][↓][削除]  │
├──────────────────────────────────────────────────────┤
│                                                      │
│ 📌 マスタデータから追加された項目                    │
│    この項目はマスタデータから追加されたため、編集できま │
│    せん。                                            │
│                                                      │
│ ID: xxx              (disabled)                      │
│ UUID: xxx-xxx-xxx    (disabled)                      │
│ Name: xxx            (disabled)                      │
│ Description: xxx     (disabled)                     │
│ Annotation: xxx      (disabled)                      │
│                                                      │
└──────────────────────────────────────────────────────┘
```

UI要素一覧

MultiObjectEditor（親コンポーネント）

1. **Collapseパネルヘッダー**
   - アイコン（objectKey別）
   - タイトル（title prop）
   - 件数Tag（成功：緑、エラー：赤）
   - 最低件数警告Tag（件数不足時のみ）

2. **マスタデータ選択セクション**（masterData.length > 0の場合のみ表示）
   - ラベル: "マスタデータから{title}を追加:"
   - Select: マスタデータ一覧（name + description表示）
   - [追加]ボタン（PlusOutlined、primary）

3. **空状態Alert**（objects.length === 0の場合）
   - 警告メッセージ: "{title}が登録されていません。最低{minItems}件必要です。"
   - タイプ: warning、showIcon

4. **オブジェクト一覧**
   - SingleObjectEditorの配列表示

5. **[+ 空の{title}を追加]ボタン**
   - タイプ: dashed
   - アイコン: PlusOutlined
   - ブロック表示

SingleObjectEditor（子コンポーネント）

1. **Collapseパネルヘッダー**
   - #番号
   - Nameタグ（name存在時）
   - IDタグ（id存在時、青色）
   - [↑]ボタン（ArrowUpOutlined、1番目は無効）
   - [↓]ボタン（ArrowDownOutlined、最後は無効）
   - [削除]ボタン（DeleteOutlined、danger、minItems以下では無効）

2. **マスタデータAlertバナー**（fromMaster === trueの場合のみ）
   - メッセージ: "📌 マスタデータから追加された項目"
   - 説明: "この項目はマスタデータから追加されたため、編集できません。"
   - タイプ: info、showIcon

3. **基本情報フィールド**（2列レイアウト）
   - **ID**: Input（small、マスタ時disabled）
   - **UUID**: Input（small、マスタ時disabled） + [🔄]ボタン（ReloadOutlined）
   - **Name**: Input（small、マスタ時disabled）
   - **Description**: Input（small、マスタ時disabled）
   - **Annotation**: TextArea（2行、small、マスタ時disabled）

4. **Referencesセクション**（includeReferences === trueの場合のみ）
   - セクション見出し: "📋 References"

   Vendor Reference（表示時）:
   - **Vendor Reference ID**: Input（placeholder: "vendorRef_example"）

- **Vendor Reference Target**: Select（vendorIds選択）

Vendor Reference（非表示時）:

- [+ Vendor Reference を追加]ボタン（block）

Instrument Reference（表示時）:

- **Instrument Reference ID**: Input（placeholder: "instrumentRef_example"）
- **Instrument Reference Target**: Select（instrumentIds選択）

Instrument Reference（非表示時）:

- [+ Instrument Reference を追加]ボタン（block）

機能仕様

Props定義

MultiObjectEditorProps（親コンポーネント）

```
interface MultiObjectEditorProps {
  objectKey: 'creators' | 'vendors' | 'owners' | 'instruments';  // オブジェクトタイプ
  title: string;                                                 // 表示タイトル
  icon: React.ReactNode;                                         // アイコン
  includeReferences?: boolean;                                   // 参照プロパティ表示フラグ
  initialData: DocumentTagData[];                                // 初期データ
  onDataChange: (                                                // データ変更コールバック
    key: 'creators' | 'vendors' | 'owners' | 'instruments',
    data: DocumentTagData[]
  ) => void;
  minItems?: number;                                             // 最低件数（デフォルト: 1）
  isExpanded: boolean;                                           // 展開状態
  onToggleExpanded: (key: string) => void;                       // 展開切り替えコールバック
  vendorIds?: string[];                                          // Vendor ID一覧（参照用）
  instrumentIds?: string[];                                      // Instrument ID一覧（参照
用）
```

```
  masterData: DocumentTagData[];                    // マスタデーター一覧
}
```

SingleObjectEditorProps（子コンポーネント）

```
interface SingleObjectEditorProps {
  index: number;                                    // インデックス番号
  objectData: DocumentTagData;                      // オブジェクトデータ
  objectKey: string;                                // オブジェクトキー
  includeReferences?: boolean;                      // 参照プロパティ表示フラグ
  onUpdate: (                                       // 更新コールバック
    index: number,
    property: string,
    value: string | Reference | null,
    type?: string
  ) => void;
  onDelete: (index: number) => void;                // 削除コールバック
  onMoveUp: (index: number) => void;                // 上移動コールバック
  onMoveDown: (index: number) => void;              // 下移動コールバック
  canMoveUp: boolean;                               // 上移動可否
  canMoveDown: boolean;                             // 下移動可否
  canDelete: boolean;                               // 削除可否
  isExpanded: boolean;                              // 展開状態
  onToggleExpanded: (index: number) => void;        // 展開切り替えコールバック
  vendorIds?: string[];                             // Vendor ID一覧
  instrumentIds?: string[];                         // Instrument ID一覧
}
```

State管理

MultiObjectEditor State

```
const [objects, setObjects] = useState<DocumentTagData[]>(initialData);
const [expandedItems, setExpandedItems] = useState<Record<number, boolean>>({});
const [selectedMasterId, setSelectedMasterId] = useState<string | undefined>(undefined);
```

SingleObjectEditor State

```
const [internalData, setInternalData] = useState<DocumentTagData>(objectData);
const updateTimeoutRef = useRef<NodeJS.Timeout | null>(null);
```

主要機能

1. 初期化処理（MultiObjectEditor）

```
const prevInitialDataRef = useRef<string>('');
useEffect(() => {
  const currentData = JSON.stringify(initialData);
  if (prevInitialDataRef.current !== currentData) {
    setObjects(initialData);
    // 最初のアイテムを展開
    if (initialData.length > 0 && Object.keys(expandedItems).length === 0) {
      setExpandedItems({ 0: true });
    }
    prevInitialDataRef.current = currentData;
  }
}, [initialData, expandedItems]);
```

動作:

- initialDataが変更されたら、内部stateを更新
- 最初のオブジェクトを自動展開
- Ref使用で変更検出（無限ループ防止）

2. マスタデータから追加

```
const handleAddFromMaster = useCallback(() => {
  if (!selectedMasterId) return;

  const masterItem = masterData.find(m => m.id === selectedMasterId);
  if (!masterItem) return;

  const newObject: DocumentTagData = {
```

```
    tagName: objectKey === 'creators' ? 'Creator' :
             objectKey === 'vendors' ? 'Vendor' :
             objectKey === 'owners' ? 'Owner' : 'Instrument',
    id: masterItem.id,
    uuid: masterItem.uuid || '',
    name: masterItem.name || '',
    description: masterItem.description || '',
    annotation: masterItem.annotation || '',
    fromMaster: true  // マスタから追加されたことをマーク
  };

  const newObjects = [...objects, newObject];
  setObjects(newObjects);
  onDataChange(objectKey, newObjects);

  // 新しいアイテムを展開
  setExpandedItems(prev => ({
    ...prev,
    [objects.length]: true
  }));

  // 選択をリセット
  setSelectedMasterId(undefined);
}, [selectedMasterId, masterData, objects, objectKey, onDataChange]);
```

動作:

1. selectedMasterIdに対応するマスタデータを検索
2. 新しいDocumentTagDataオブジェクトを作成
3. `fromMaster: true` フラグを設定（編集不可マーク）
4. objectsに追加してonDataChangeコールバック呼び出し
5. 新しいアイテムを自動展開
6. Select選択をリセット

3. 空のオブジェクト追加

```
const handleAdd = useCallback(() => {
  const newObject: DocumentTagData = {
    tagName: objectKey === 'creators' ? 'Creator' :
             objectKey === 'vendors' ? 'Vendor' :
             objectKey === 'owners' ? 'Owner' : 'Instrument',
    id: `${objectKey}${objects.length + 1}`,
```

```
    uuid: '',
    name: '',
    description: '',
    annotation: ''
  };

  const newObjects = [...objects, newObject];
  setObjects(newObjects);
  onDataChange(objectKey, newObjects);

  // 新しいアイテムを展開
  setExpandedItems(prev => ({
    ...prev,
    [objects.length]: true
  }));
}, [objects, objectKey, onDataChange]);
```

動作:

1. objectKeyに応じてtagNameを設定
2. デフォルトID（`{objectKey}{番号}`）を生成
3. 空のオブジェクトを作成してobjectsに追加
4. 新しいアイテムを自動展開

4. オブジェクト削除

```
const handleDelete = useCallback((index: number) => {
  const newObjects = objects.filter((_, i) => i !== index);
  setObjects(newObjects);
  onDataChange(objectKey, newObjects);
}, [objects, objectKey, onDataChange]);
```

動作:

- 指定インデックスのオブジェクトをfilterで除外
- 確認ダイアログはSingleObjectEditor側で実施
- minItems未満になる場合は削除ボタンが無効化される

5. オブジェクト更新

```
const handleUpdate = useCallback((
  index: number,
  property: string,
  value: string | Reference | null,
  type: string = 'simple'
) => {
  const newObjects = [...objects];

  if (type === 'reference_delete') {
    if (newObjects[index][property as keyof DocumentTagData]) {
      delete newObjects[index][property as keyof DocumentTagData];
    }
  } else if (type === 'reference') {
    newObjects[index] = {
      ...newObjects[index],
      [property]: value
    };
  } else {
    newObjects[index] = {
      ...newObjects[index],
      [property]: value
    };
  }

  setObjects(newObjects);
  onDataChange(objectKey, newObjects);
}, [objects, objectKey, onDataChange]);
```

更新タイプ:

- `'simple'`: 通常のプロパティ更新
- `'reference'`: Reference型プロパティ更新
- `'reference_delete'`: Referenceプロパティ削除（id/refが両方空の場合）

6. 並び替え（上/下移動）

```
// 上に移動
const handleMoveUp = useCallback((index: number) => {
  if (index === 0) return;

  const newObjects = [...objects];
  [newObjects[index - 1], newObjects[index]] = [newObjects[index], newObjects[index - 1]];
```

```
  setObjects(newObjects);
  onDataChange(objectKey, newObjects);
}, [objects, objectKey, onDataChange]);

// 下に移動
const handleMoveDown = useCallback((index: number) => {
  if (index === objects.length - 1) return;

  const newObjects = [...objects];
  [newObjects[index], newObjects[index + 1]] = [newObjects[index + 1], newObjects[index]];
  setObjects(newObjects);
  onDataChange(objectKey, newObjects);
}, [objects, objectKey, onDataChange]);
```

動作:

- 分割代入で隣接要素とスワップ
- 先頭/末尾では実行しない（ボタンは無効化）

7. 遅延更新（SingleObjectEditor - Debounce機能）

```
const handleInternalUpdate = useCallback((property: string, value: string) => {
  // ✅ 即座にUI更新（レスポンス向上）
  setInternalData(prev => ({
    ...prev,
    [property]: value
  }));

  // ✅ 500ms遅延で親に通知（API呼び出し削減）
  if (updateTimeoutRef.current) {
    clearTimeout(updateTimeoutRef.current);
  }
  updateTimeoutRef.current = setTimeout(() => {
    onUpdate(index, property, value);
  }, 500);
}, [index, onUpdate]);
```

メリット:

- ユーザーがタイピング中は親への通知を遅延
- 最終的な値だけを親に伝達（パフォーマンス向上）

- UIは即座に更新（レスポンス良好）

8. UUID自動生成

```
const handleUUIDGenerate = useCallback(() => {
  const newUUID = 'xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx'.replace(/[xy]/g, function(c) {
    const r = Math.random() * 16 | 0;
    const v = c === 'x' ? r : (r & 0x3 | 0x8);
    return v.toString(16);
  });
  handleInternalUpdate('uuid', newUUID);
}, [handleInternalUpdate]);
```

動作:

- RFC 4122準拠のUUID v4を生成
- [🔄]ボタンクリックで実行
- 遅延更新で親に通知

9. 参照プロパティ更新

```
const handleReferenceUpdate = useCallback((
  refKey: 'vendorRef' | 'instrumentRef',
  property: 'id' | 'ref',
  value: string
) => {
  // ✅ 空の場合は削除処理
  if (!value || value.trim() === '') {
    setInternalData(prev => {
      const newData = { ...prev };
      if (newData[refKey]) {
        const updatedRef : Reference = { ...newData[refKey] };
        if (property === 'id') {
          updatedRef.id = '';
        } else {
          updatedRef.ref = '';
        }

        // 両方空なら、refKeyごと削除
        if (!updatedRef.id && !updatedRef.ref) {
          const { [refKey]: _, ...rest } = newData;
```

```
          return rest;
        }

        newData[refKey] = updatedRef;
      }
      return newData;
    });

    // 親に削除通知
    if (updateTimeoutRef.current) {
      clearTimeout(updateTimeoutRef.current);
    }
    updateTimeoutRef.current = setTimeout(() => {
      onUpdate(index, refKey, null, 'reference_delete');
    }, 500);

    return;
  }

  // ✅ 通常の更新処理
  setInternalData(prev => ({
    ...prev,
    [refKey]: {
      ...(prev[refKey] || { id: '', ref: '' }),
      [property]: value
    }
  }));

  // 親に更新通知
  if (updateTimeoutRef.current) {
    clearTimeout(updateTimeoutRef.current);
  }
  updateTimeoutRef.current = setTimeout(() => {
    const updatedRef: Reference = {
      ...(internalData[refKey] || { id: '', ref: '' }),
      [property]: value
    };
    onUpdate(index, refKey, updatedRef, 'reference');
  }, 500);
}, [index, onUpdate, internalData]);
```

動作:

1. 空値の場合は削除処理
2. id/refが両方空ならrefKeyプロパティごと削除
3. 通常値の場合はReferenceオブジェクトを更新

4. 500ms遅延で親に通知

10. Reference ID連番生成

```
const generateReferenceId = useCallback((refType: 'vendorRef' | 'instrumentRef') => {
  const existingRefs = internalData[refType] ? 1 : 0;
  const basePrefix = refType === 'vendorRef' ? 'vendorRef' : 'instrumentRef';
  const nextNumber = existingRefs + 1;
  return `${basePrefix}_${nextNumber}`;
}, [internalData]);
```

動作:

- `vendorRef_1`、`instrumentRef_1`形式のIDを生成
- [追加]ボタンクリック時にid欄に自動設定

データ型定義

DocumentTagData

```
interface DocumentTagData {
  tagName: string;                    // タグ名（Creator/Vendor/Owner/Instrument）
  id?: string;                        // ID
  uuid?: string;                      // UUID
  name?: string;                      // 名前
  description?: string;               // 説明
  annotation?: string;                // 注釈
  vendorRef?: Reference;              // Vendor参照（Instrumentのみ）
  instrumentRef?: Reference;          // Instrument参照（Ownerのみ）
  fromMaster?: boolean;               // マスタデータから追加されたフラグ
  [key: string]: any;                 // その他の動的プロパティ
}
```

Reference

```
interface Reference {
  id?: string;                          // Reference ID
  ref: string;                          // 参照先ID
}
```

関連コンポーネント

親コンポーネント

DocumentEditDialog

- **ファイルパス**: `frontend/src/components/editorv2/components/DocumentEditDialog.tsx`
- **用途**: MAIML Document編集ダイアログ
- **連携**:
  - Creators/Vendors/Owners/Instrumentsの編集にMultiObjectEditorを使用
  - onDataChangeコールバックでDocument構造を更新

子コンポーネント

SingleObjectEditor

- **位置**: 同一ファイル内
- **用途**: 個別オブジェクトの詳細編集
- **Props**:
  - index、objectData、onUpdate、onDelete、onMoveUp、onMoveDown等
  - 展開状態、並び替えボタンの有効/無効制御

使用コンポーネント（Ant Design）

- **Collapse/Panel**: 折りたたみ可能なパネル
- **Input**: テキスト入力
- **TextArea**: 複数行テキスト入力
- **Select**: ドロップダウン選択
- **Button**: ボタン
- **Tag**: タグ表示
- **Alert**: 警告・情報メッセージ
- **Space**: 要素間のスペース調整
- **Row/Col**: グリッドレイアウト

パフォーマンス最適化

1. 遅延更新（Debounce）

```
const updateTimeoutRef = useRef<NodeJS.Timeout | null>(null);

// 500ms debounce
setTimeout(() => {
  onUpdate(index, property, value);
}, 500);
```

メリット:

- タイピング中の不要な親更新を削減
- API呼び出し回数の削減
- UIレスポンスの向上

2. useCallback使用

すべてのハンドラー関数で `useCallback` を使用し、不要な再生成を防止：

- `handleAdd`、`handleDelete`、`handleUpdate`
- `handleMoveUp`、`handleMoveDown`

- `handleAddFromMaster`
- `handleInternalUpdate`、`handleReferenceUpdate`

3. Ref使用の変更検出

```
const prevInitialDataRef = useRef<string>('');
useEffect(() => {
  const currentData = JSON.stringify(initialData);
  if (prevInitialDataRef.current !== currentData) {
    // 変更があった場合のみ更新
  }
}, [initialData]);
```

メリット:

- useEffectの無限ループを防止
- 実際に変更があった場合のみState更新

4. クリーンアップ処理

```
useEffect(() => {
  return () => {
    if (updateTimeoutRef.current) {
      clearTimeout(updateTimeoutRef.current);
    }
  };
}, []);
```

動作:

- コンポーネントアンマウント時にタイマーをクリア
- メモリリーク防止

エラーハンドリング

最低件数チェック

```
{objects.length < minItems && (
  <Tag color="warning">最低{minItems}件必要</Tag>
)}

{objects.length === 0 && (
  <Alert
    message={`${title}が登録されていません。最低${minItems}件必要です。`}
    type="warning"
    showIcon
  />
)}
```

削除確認ダイアログ

```
onClick={(e) => {
  e.stopPropagation();
  if (window.confirm('この要素を削除しますか？')) {
    onDelete(index);
  }
}}
```

マスタデータ編集防止

```
{internalData.fromMaster && (
  <Alert
    message="📌 マスタデータから追加された項目"
    description="この項目はマスタデータから追加されたため、編集できません。"
    type="info"
    showIcon
  />
)}

<Input
  disabled={internalData.fromMaster}  // マスタから追加された項目は編集不可
/>
```

使用例

DocumentEditDialog内での使用

```
<MultiObjectEditor
  objectKey="creators"
  title="Creators"
  icon={<TeamOutlined />}
  includeReferences={false}
  initialData={creators}
  onDataChange={handleCreatorsChange}
  minItems={1}
  isExpanded={expandedSections.creators}
  onToggleExpanded={handleToggleSection}
  masterData={masterCreators}
/>

<MultiObjectEditor
  objectKey="instruments"
  title="Instruments"
  icon={<ExperimentOutlined />}
  includeReferences={true}  // Vendor/Instrument参照を表示
  initialData={instruments}
  onDataChange={handleInstrumentsChange}
  minItems={1}
  isExpanded={expandedSections.instruments}
  onToggleExpanded={handleToggleSection}
  vendorIds={vendorIds}  // 参照先Vendor一覧
  masterData={masterInstruments}
/>
```

変更履歴

日付	バージョン	変更内容	担当者
2026-03-12	1.0.0	実装ベースでUI仕様書を新規作成	Cline
2026-02-02	-	TypeScript + Ant Design実装完了（実装）	-
2025-XX-XX	-	複数オブジェクト編集コンポーネント初版作成（実装）	-

実装ファイル情報

- **ファイルパス**: `frontend/src/components/editorv2/components/MultiObjectEditor.tsx`
- **コンポーネント名**: `MultiObjectEditor` / `SingleObjectEditor`
- **実装バージョン**: 2.0.0
- **実装日**: 2026-02-02